```
Script started on 2026-02-07 01:02:18-06:00 [TERM="xterm" TTY="/dev/pts/27" COLUMNS
rc47379@ares:~$ pwd

/home/students/rc47379
rc47379@ares:~$ cat imaginary.info

NAME: Calvin Rice

CLASS: CSC122-002

TITLE: What a complex problem

LEVEL: 3

DESCRIPTION:

A class that does various math operations on complex numbers with real and imaginar



imaginary.cpp:
```

```cpp
#include <iostream>
#include <cmath>

class ComplexNumber
{
private:
  double real, imaginary;
public:
  ComplexNumber(double _real, double _imaginary)
  {
    real = _real;
    imaginary = _imaginary;
  }

  ComplexNumber add(ComplexNumber _c)
  {
    return ComplexNumber(real + _c.real, imaginary + _c.imaginary);
  }

  ComplexNumber subtract(ComplexNumber _c)
  {
    return ComplexNumber(real - _c.real, imaginary - _c.imaginary);
  }

  ComplexNumber multiply(ComplexNumber _c)
  {
    return ComplexNumber((real * _c.real) - (imaginary * _c.imaginary), (re
  }

  ComplexNumber divide(ComplexNumber _c)
  {
    double denominator = (_c.real * _c.real) + (_c.imaginary * _c.imaginary
    if(denominator == 0.0)
    {
      return ComplexNumber(0, 0);
    }

    double newReal = (real * _c.real + imaginary * _c.imaginary) / denomina
    double newImaginary = (imaginary * _c.real - real * _c.imaginary) / den
    return ComplexNumber(newReal, newImaginary);
  }

  double magnitude()
  {
    return sqrt(real * real + imaginary * imaginary);
  }

  ComplexNumber negate()
  {
    return ComplexNumber(-real, -imaginary);
  }

  ComplexNumber conjugate()
  {
    return ComplexNumber(real, -imaginary);
  }

  double get_real()
  {
    return real;
  }

  double get_imaginary()
  {
    return imaginary;
  }

  void output()
  {
    std::cout << real << " + " << imaginary << "i";
  }
};

int main() {
  ComplexNumber a(4, 3);
  ComplexNumber b(1, -2);

  std::cout << "complex number a = ";
  a.output();

  std::cout << "\ncomplex number b = ";
  b.output();

  std::cout << "\na + b = ";
  a.add(b).output();
```

```
    86
    87    std::cout << "\na - b = ";
    88    a.subtract(b).output();
    89
    90    std::cout << "\na * b = ";
    91    a.multiply(b).output();
    92
    93    std::cout << "\na / b = ";
    94    a.divide(b).output();
    95
    96    std::cout << "\nmagnitude(a) = " << a.magnitude() << std::endl;
    97
    98    std::cout << "negate(a) = ";
    99    a.negate().output();
   100
   101    std::cout << "\nconjugate(a) = ";
   102    a.conjugate().output();
   103
   104    std::cout << "\nreal part of a: " << a.get_real() << std::endl;
   105    std::cout << "imaginary part of a: " << a.get_imaginary() << std::endl;
   106
   107    return 0;
   108  }
rc47379@ares:~$ CPP imaginary

imaginary.cpp***
./imaginary.cpp: In constructor
‘ComplexNumber::ComplexNumber(double,
double)’:
imaginary.cpp:9:3: warning:
‘ComplexNumber::real’ should be initialized
in the member initialization list [-Weffc++]
    9 |   ComplexNumber(double _real, double
    _imaginary)
      |   ^~~~~~~~~~~~
imaginary.cpp:9:3: warning:
‘ComplexNumber::imaginary’
should be initialized in the member initialization list
[-Weffc++]
imaginary.cpp: In member
function ‘ComplexNumber
ComplexNumber::divide(ComplexNumber)’:
imaginary.cpp:33:20: warning: comparing
floating-point with ‘==’ or ‘!=’
is unsafe [-Wfloat-equal]
   33 |     if(denominator == 0.0)
      |        ~~~~~~~~~~~~^~~~~


rc47379@ares:~$ ./imaginary.out

complex number a = 4 + 3i
complex number b = 1 + -2i
a + b = 5 + 1i
a - b = 3 + 5i
a * b = 10 + -5i
a / b = -0.4 + 2.2i
magnitude(a) = 5
negate(a) = -4 + -3i
conjugate(a) = 4 + -3i
real part of a: 4
imaginary part of a: 3
rc47379@ares:~$ cat imaginary.tpq

Why do your class methods take fewer arguments than you would expect?

-They only take one complex number that is being operated on with the calling objec

Does the compiler change ywhen you have 'x+ y' in your program? So should your add

-The program makes a new object to output so neither inputs are changed

Does this phenomenon extend to the other operations?

-Yes

What kind of value should be returned from the standard math operations (i.e. what

-Complex numbers should be returned from any method that keeps i, and the magnitude

Does your inputmethod prompt the user? Why should it not?

-My input method doesn't exist becuase there is no reason for the user to input sor

Does your outputmethod print anything besides the Complexnumber (even an endl)? Why

-It shouldn't because the user might want a different formatted outputrc47379@ares

exit

Script done on 2026-02-07 01:02:53-06:00 [COMMAND_EXIT_CODE="0"]
```